

DEVADN

Sécurité serveur

Durcissement SSH et fail2ban

De l'installation à l'utilisation

1 août 2026

Sommaire

1. Durcissement SSH et fail2ban	3
La règle d'or	3
Partie 1 — N'autoriser que l'authentification par clé	3
Partie 2 — fail2ban	5
Partie 3 — Vérifications régulières	6
Ce qui reste à faire	7
À retenir	7

CHAPITRE 1

Durcissement SSH et fail2ban

Procédure appliquée au VPS de production, avec les pièges rencontrés.

Ce document n'est pas un tutoriel générique : il décrit ce qui s'est réellement passé sur ce serveur, y compris deux configurations qui donnaient l'illusion d'être protégé sans l'être. Les commandes sont à exécuter en SSH sur le VPS.

La règle d'or

Garde toujours ta session SSH actuelle ouverte pendant toute manipulation du démon SSH, et teste depuis une **seconde fenêtre**.

Si tu te verrouilles dehors, la session déjà ouverte est ton seul moyen de revenir en arrière. Une fois fermée, il ne reste que la console de secours de l'hébergeur. Ce n'est pas une précaution théorique : la configuration SSH est la seule qu'on ne peut pas corriger à distance après l'avoir cassée.

Partie 1 — N'autoriser que l'authentification par clé

L'objectif est d'empêcher toute connexion par mot de passe. Un mot de passe se devine, se rejoue, se retrouve dans une fuite ; une clé, non.

Étape 1 : vérifier que ta clé fonctionne AVANT tout

Ne désactive jamais les mots de passe avant d'avoir prouvé que ta clé passe.

```
wc -l < ~/.ssh/authorized_keys
```

Si le compte est à zéro, ajoute d'abord ta clé publique depuis ton poste :

```
ssh-copy-id -p 49231 deploy@devadn.com
```

Puis, depuis une nouvelle fenêtre, teste une connexion strictement par clé :

```
ssh -o PasswordAuthentication=no -p 49231 deploy@devadn.com "echo CLE_OK"
```

Ne continue que si **CLE_OK** s'affiche.

Étape 2 : le piège du répertoire sshd_config.d

C'est ici que le durcissement échouait silencieusement sur ce serveur.

Le fichier `/etc/ssh/sshd_config` commence par une directive **Include** qui charge tous les fichiers `.conf` de `/etc/ssh/sshd_config.d/`, dans l'ordre alphabétique. Sur ce VPS ils contenaient :

- `50-cloud-init.conf` : `PasswordAuthentication yes`
- `60-cloudimg-settings.conf` : `PasswordAuthentication no`
- `99-disable-password.conf` : `PasswordAuthentication no`

Sur OpenSSH, c'est la PREMIÈRE valeur rencontrée qui gagne, pas la dernière. C'est l'inverse de l'intuition et de la plupart des autres logiciels de configuration.

Le `50-cloud-init.conf`, déposé par l'hébergeur à la création du serveur, imposait donc **yes** à lui seul. Les deux fichiers suivants qui disaient **no** étaient purement décoratifs. Un durcissement avait d'ailleurs déjà été tenté des mois plus tôt, sans jamais avoir eu le moindre effet.

La leçon : ne modifie pas `sshd_config` sans regarder ce que le répertoire `sshd_config.d` contient déjà.

```
sudo grep -rn "" /etc/ssh/sshd_config.d/
```

Étape 3 : poser un fichier qui gagne

Puisque le premier fichier l'emporte, on en crée un qui passe avant tous les autres, avec un préfixe numérique bas :

```
printf 'PasswordAuthentication no\nKbdInteractiveAuthentication\n\nPubkeyAuthentication yes\nPermitRootLogin prohibit-password\n' | sudo tee /etc/ssh/sshd_config.d/00-hardening.conf
```

`KbdInteractiveAuthentication no` n'est pas superflu : sans cette ligne, le module PAM peut réautoriser une saisie de mot de passe malgré `PasswordAuthentication no`. C'est un contournement classique.

On ne touche pas au `Port` : celui défini plus loin reste actif.

Corrige aussi le fichier fautif, pour que la configuration reste cohérente même si l'ordre de lecture change un jour :

```
sudo sed -i 's/^PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config.d/50-cloud-init.conf
```

Enfin, empêche cloud-init de le réécrire au prochain redémarrage :

```
echo 'ssh_pwauth: false' | sudo tee /etc/cloud/cloud.cfg.d/99-disable-ssh-pwauth.cfg
```

Étape 4 : reload ne suffit pas, il faut restart

Deuxième piège rencontré. Un `systemctl reload ssh` avait bien envoyé le signal au démon, mais celui-ci n'a pas pu reprendre le port :

```
error: Bind to port 49231 on 0.0.0.0 failed: Address already in use
```

Le démon a continué de tourner **avec l'ancienne configuration en mémoire**. Vu de l'extérieur, tout semblait normal. Le service affichait onze heures de fonctionnement ininterrompu, ce qui était l'indice : un rechargement réussi n'aurait rien changé à ce compteur, mais la configuration, elle, n'avait pas bougé non plus.

Vérifie d'abord la configuration effective :

```
sudo sshd -T | grep -iE "(passwordauthentication|kbdinteractiveauthentication|permitrootlogin|port) "
```

Tu dois lire `passwordauthentication no`. Valide la syntaxe, puis redémarre réellement :

```
sudo sshd -t && sudo systemctl restart ssh && systemctl is-active ssh
```

Le redémarrage ne coupe pas les sessions en cours : chaque connexion est gérée par un processus enfant distinct.

Étape 5 : les deux tests qui font foi

Depuis une nouvelle fenêtre, sans fermer la session courante.

Le mot de passe doit être refusé :

```
ssh -o PubkeyAuthentication=no -o PreferredAuthentications=password -p 49231 deploy@devadn.com
```

Résultat attendu : `Permission denied (publickey)`.

La clé doit toujours passer :

```
ssh -p 49231 deploy@devadn.com "echo ACCES_OK"
```

Étape 6 : ne pas oublier l'automatisation

Le déploiement GitHub Actions se connecte lui aussi en SSH. Si sa clé était affectée, on ne s'en apercevrait qu'au déploiement suivant, au pire moment.

Relance un déploiement après le durcissement et vérifie qu'il aboutit. Sur ce serveur, le contrôle a été fait : déploiement réussi en 39 secondes après désactivation des mots de passe.

Revenir en arrière

Si quelque chose se passe mal et que la session ouverte fonctionne encore :

```
sudo rm /etc/ssh/sshd_config.d/00-hardening.conf && sudo systemctl restart ssh
```

Partie 2 — fail2ban

fail2ban lit les journaux d'authentification et bannit temporairement les adresses qui multiplient les échecs. Avec l'authentification par clé seule, les tentatives par mot de passe échouent déjà toutes, mais fail2ban évite que ces tentatives saturent le serveur et réduit le bruit dans les journaux.

Installation

```
sudo apt update && sudo apt install -y fail2ban
```

Les deux réglages indispensables sur ce serveur

La configuration par défaut ne fonctionnerait pas ici, pour deux raisons.

Le port n'est pas standard. La prison `sshd` par défaut surveille le port

22. Le SSH de ce serveur écoute sur 49231 : sans le préciser, fail2ban tournerait sans jamais rien bannir.

Ubuntu 24.04 n'écrit plus `/var/log/auth.log`. Les journaux passent par journald. Une prison qui lit un fichier inexistant reste silencieuse. Le symptôme est reconnaissable : `tail /var/log/auth.log` ne renvoie rien.

On écrit donc une configuration explicite dans `jail.local` — jamais dans `jail.conf`, qui est écrasé à chaque mise à jour du paquet :

```
sudo tee /etc/fail2ban/jail.local > /dev/null <<'EOF'
[DEFAULT]
bantime = 1h
findtime = 10m
maxretry = 5

[sshd]
enabled = true
port = 49231
backend = systemd
EOF
```

Trois échecs suffisent rarement à distinguer un attaquant d'un utilisateur distrait ; cinq tentatives en dix minutes, bannies une heure, est un réglage raisonnable pour un serveur à un seul administrateur.

Activer et vérifier

```
sudo systemctl enable --now fail2ban
```

```
sudo fail2ban-client status sshd
```

La sortie doit indiquer la prison active, le nombre d'échecs détectés et la liste des adresses bannies. **Si la prison n'apparaît pas**, la configuration n'est pas valide :

```
sudo fail2ban-client -d | head -20
```

Voir les tentatives

C'est la question qu'on se pose une fois fail2ban en place : est-ce que quelqu'un essaie vraiment ?

Le tableau de bord de la prison donne l'essentiel — échecs en cours, total cumulé depuis le démarrage, adresses bannies, total des bannissements :

```
sudo fail2ban-client status sshd
```

L'historique des bannissements et déblocages :

```
sudo journalctl -u fail2ban --since "7 days ago" | grep -iE "\bBan\b|\bUnban\b"
```

Les tentatives brutes, y compris celles qui n'ont pas atteint le seuil de bannissement :

```
sudo journalctl -u ssh --since "24 hours ago" | grep -i "failed|invalid user" | tail -30
```

Le classement des adresses les plus insistantes, pour distinguer un balayage automatisé d'une cible qui revient :

```
sudo journalctl -u ssh --since "7 days ago" | grep -oE "from [0-9a-f.:]+" | sort | uniq -c | sort -rn | head -10
```

Un serveur exposé sur Internet reçoit des tentatives en permanence : des milliers par semaine sur le port 22, beaucoup moins sur un port non standard. Leur présence n'a rien d'alarmant. Ce qui compte, c'est qu'aucune n'aboutisse — et avec l'authentification par clé seule, aucune ne le peut.

Débannir une adresse

Utile si tu te bannis toi-même depuis un réseau public :

```
sudo fail2ban-client set sshd unbanip 203.0.113.10
```

Pour ne jamais bannir une adresse de confiance, ajoute-la dans la section [\[DEFAULT\]](#) de [jail.local](#) :

```
ignoreip = 127.0.0.1/8 ::1 203.0.113.10
```

Partie 3 — Vérifications régulières

Consulter les connexions acceptées et refusées, sur un système journald :

```
sudo journalctl -u ssh --since "24 hours ago" | grep -iE "accepted|failed" | tail -20
```

Voir les adresses actuellement bannies :

```
sudo fail2ban-client status sshd
```

Vérifier que la configuration SSH effective n'a pas dérivé, notamment après une mise à jour système ou une intervention de l'hébergeur :

```
sudo sshd -T | grep -i "^passwordauthentication "
```

Cette dernière commande mérite d'être relancée périodiquement : c'est exactement le contrôle qui aurait révélé, des mois plus tôt, que le durcissement précédent n'avait jamais pris effet.

Ce qui reste à faire

Deux mesures complémentaires, non couvertes par cette procédure.

Le pare-feu. N'ouvrir que le port SSH et les ports web. Autoriser impérativement le port SSH **avant** d'activer le pare-feu, sous peine de se couper soi-même.

```
sudo ufw allow 49231/tcp comment SSH && sudo ufw allow 80/tcp && sudo ufw allow 443/tcp
```

```
sudo ufw --force enable && sudo ufw status numbered
```

Le conteneur applicatif écoute sur [127.0.0.1:3000](#) et n'est donc pas exposé : rien d'autre à ouvrir.

Les mises à jour de sécurité automatiques.

```
sudo apt install -y unattended-upgrades && sudo dpkg-reconfigure -plow unattended-upgrades
```

À retenir

Trois enseignements de cette intervention, valables au-delà de SSH.

Une configuration écrite n'est pas une configuration appliquée. Le fichier disait [no](#) depuis des mois ; le serveur acceptait les mots de passe. Seul un test réel depuis l'extérieur fait foi.

L'ordre de priorité n'est pas toujours celui qu'on croit. Sur OpenSSH, le premier fichier lu gagne. Un correctif ajouté « à la fin » peut n'avoir aucun effet.

Toujours vérifier ce qui dépend de ce qu'on modifie. Le durcissement SSH touchait aussi le déploiement automatique. Ne pas le tester, c'était choisir de découvrir le problème au prochain déploiement urgent.